Are LLM Written Unit Tests Flaky?

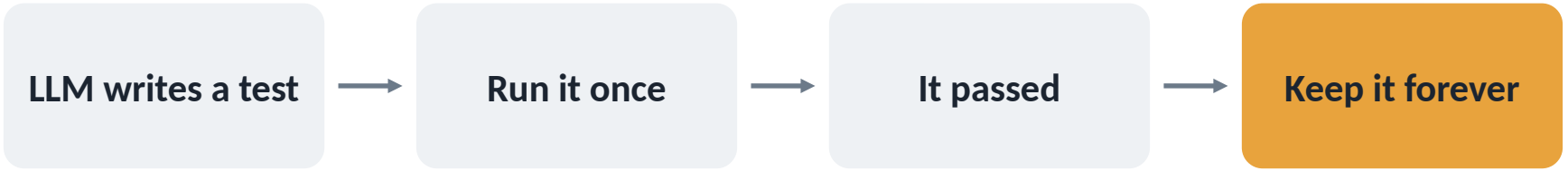
Initial project idea for ECE 382V

Shaunak Kapur, working individually



30 reruns of one test, one of them failed

Generation pipelines trust one passing run



- TestPilot and most Copilot style pipelines keep a generated test if it passes one run.
- Every big flaky test study so far looks at tests written by people.
- So I want to know if LLM tests are more flaky, why, and if one run is enough to catch it.



A flaky test can pass the one run that decides if it gets kept.

What I am running

12 Python libraries

cachetools	schedule	tinydb
humanize	toolz	tabulate
tenacity	filelock	boltons
more-itertools	arrow	pybreaker

Tan ones were added because they use time, randomness, threads, or files.

4 test suites per library

Human
the library's own test suite

Agentic Sonnet
runs its tests and fixes failures

One shot Sonnet
writes tests, never runs them

One shot Haiku
same, with a smaller model

Every suite reruns 70 times with

- random order and hash seed
- 10 time zones
- CPU stress on every core

Early results

714,000

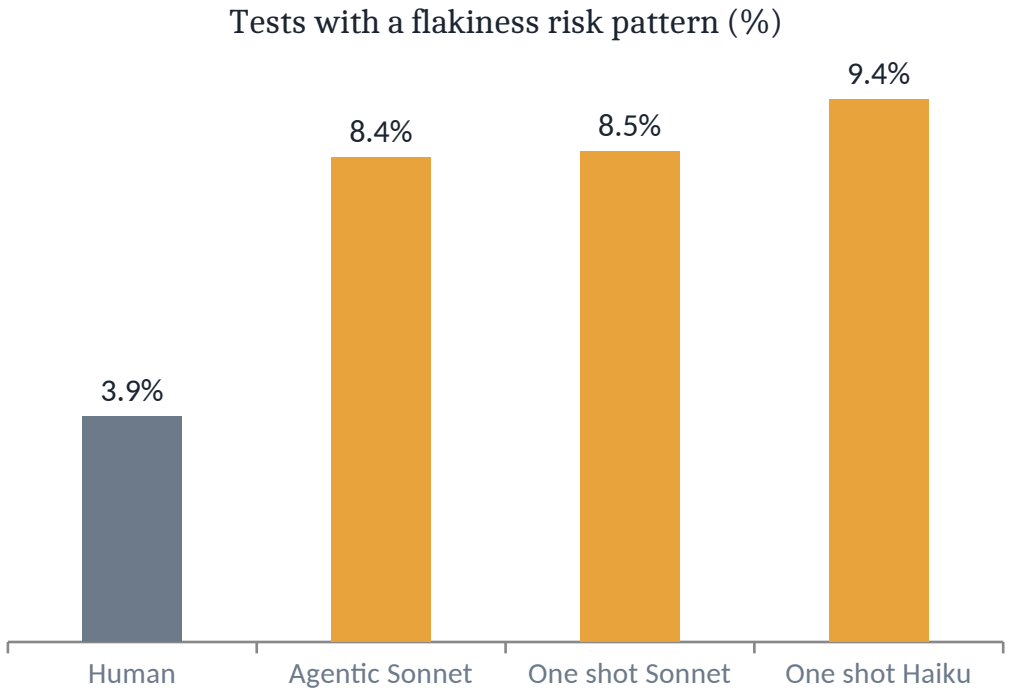
test executions across 48 suites

3

LLM tests flipped, all about real time, and a one run filter kept all of them

6

human tests flipped, mostly because of test order



Next steps and open questions

How I will evaluate

- Flaky rate per suite with confidence intervals
- Root cause of every flaky test
- Risk patterns and coverage per suite
- What a one run filter keeps and throws away

Timeline

- Oct 9 proposal
- Oct 24 more models and reruns
- Nov 7 slower CI machine runs
- Nov 21 mitigation prompt
- Dec 12 final report

What I am unsure about

- Flips are rare, so do I need 1000 reruns
- Do risk patterns predict real flips
- Is a laptop too fast to show timing bugs
- Ideas for other perturbations welcome